



SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF INGEGNERIA INFORMATICA, AUTOMATICA E
GESTIONALE

Space Valley Project

INTERACTIVE GRAPHICS

Professor:
Marco Schaerf

Students:
Antonio Accardo
2279885
Francesco Morazio
2018794

Contents

1	Introduction	2
2	Environment and Tools	2
2.1	Libraries and External Tools	2
2.2	External Assets	2
2.3	Project Architecture	3
3	Hierarchical Models	3
3.1	Character Hierarchy	4
3.2	Static Obstacles	4
4	Lighting and Textures	4
4.1	Lighting	4
4.2	Textures	5
5	User Interaction	5
6	Animations	6
6.1	Locomotion Animation	6
6.2	Body Turning Animation	7
6.3	Throwing Animation	7
6.4	Day/Night Cycle	7
6.5	Procedural Sky Rotation	7
7	Physics and Collision System	8
7.1	Planet's Gravity	8
7.2	Jumping and Ground Detection	8
7.3	Obstacle Collision	8
7.4	Camera Collision	8
7.5	Spheres Physics	9

1 Introduction

This project is an interactive 3D exploration experience built with Three.js, in which the player controls an astronaut-like character able to walk, jump, look around and throw objects across the surface of different planets and moons of the Solar System. The core idea of the project is to let the same character and the same set of mechanics behave differently depending on the celestial body currently selected by the user: gravity, movement speed, jump height, walking animation and visual appearance of the sky and terrain all change consistently with the physical properties of the chosen planet.

The player can switch between nine different locations: Mercury, Venus, Earth, the Moon, Mars, Jupiter, Saturn, Uranus and Neptune. This selection is done through a simple UI selector. Each planet has its own gravitational acceleration, its own textured terrain, its own atmospheric color and fog behavior, and its own star visibility. Earth is treated as a special case, featuring a fully dynamic day/night cycle that continuously changes the sky color and star opacity based on the position of the sun.

The project combines several classic elements studied during by the course – hierarchical models, lighting, textures, user interaction and custom animations – with an additional custom-built physics and collision layer implemented to make movement, jumping and object throwing feel physically consistent across very different gravities (from Jupiter’s 24.79m/s^2 down to the Moon’s 1.62m/s^2).

2 Environment and Tools

The project is developed entirely in JavaScript using **Three.js** as the WebGL rendering library. No other external physics or animation engine is used: all gravity simulation, collision detection and animation logic were implemented manually.

2.1 Libraries and External Tools

- **Three.js** is used for scene management, the WebGL renderer, cameras, geometries, materials, lights and the texture loading pipeline.
- No physics engine is used. All physics (gravity, jumping, collision resolution, projectile motion) is implemented from scratch in the **Physics** and **WeaponSystem** modules.

2.2 External Assets

The terrain of each planet uses a set of external texture maps (color, normal and roughness) stored per-planet under an `assets/textures/{planet}/` folder structure.

No external 3D models were imported: the player character and all environmental obstacles are procedurally built using primitive Three.js geometries (boxes and cylinders) combined into hierarchical groups.

2.3 Project Architecture

The codebase is organized into independent modules, each responsible for a single aspect of the simulation:

- `main.js` is the entry point and the main render/update loop.
- `World.js` is responsible of terrain generation, planet textures and static obstacles.
- `Physics.js` handles gravity, jumping and collision resolution.
- `Player.js` implements character model and locomotion animation.
- `Input.js` controls keyboard and mouse input handling.
- `MouseControls.js` takes care of pointer-lock based look controls.
- `CameraManager.js` manages first/third person camera logic and camera collision.
- `WeaponSystem.js` tackles projectile spawning and physics.
- `DayNightSystem.js` computes sun position and day/night lighting cycle.
- `SkyboxManager.js` handles procedural starfield and atmospheric color per planet.

This modular design keeps rendering, physics, input and animation logic clearly separated, and allows each planet-dependent behavior (gravity, textures, sky color) to be updated independently through a single custom `planetChanged` event dispatched whenever the user selects a new planet from the UI.

3 Hierarchical Models

The main hierarchical model in the project is the player character, built in `Player.js` entirely out of primitive geometries organized in a parent-child tree so that each body part can be animated independently while still moving coherently with the rest of the body.

3.1 Character Hierarchy

The character is rooted in a `THREE.Group` (`this.group`) that represents the character's position in the world and is the object actually moved by the physics system. Inside this root group:

- The **torso** is a box mesh attached directly to the root group, and acts as the parent for almost every other body part.
- The **head** (a smaller box) is attached as a child of the torso, and a **visor** (a thin colored box) is in turn attached as a child of the head. This creates a three-level hierarchy (group → torso → head → visor) in which moving or rotating the torso automatically carries the head and visor with it.
- Both **arms** are attached through dedicated pivot groups (`rightArmPivot` and `leftArmPivot`) positioned at the shoulder height on the torso. The actual arm mesh is offset downward inside its pivot, so that rotating the pivot swings the arm around the shoulder joint rather than around the arm's own center – exactly as a real limb would rotate around its joint.
- Both **legs** follow the same pattern, with `rightLegPivot` and `leftLegPivot` attached to the torso at hip height, each containing a leg mesh offset downward from the pivot.

This hierarchical construction is what allows the walking animation to rotate only the local pivot of each limb, letting Three.js's scene graph automatically propagate the correct world-space transform to every mesh, without the animation code needing to manually compute forward kinematics.

3.2 Static Obstacles

A secondary, simpler set of hierarchical/structural elements is the field of static obstacles generated in `World.js`. Obstacles are categorized into three visual/functional classes – tall towers, medium crates and low obstacles – using box and cylinder geometries. Although these are not deeply nested hierarchies, their axis-aligned bounding boxes are precomputed once at creation time and reused every frame by the collision system avoiding expensive per-frame bounding box recalculation.

4 Lighting and Textures

4.1 Lighting

The scene lighting is centered around a dynamic directional light representing the sun, managed by the `DayNightSystem` module. The sun's position, color and intensity are

updated every frame based on the currently selected planet: each planet has its own configured sun color and intensity, reflecting the real relative distances of the planets from the sun.

On Earth specifically, the sun’s vertical position drives a continuous day/night lighting cycles.

4.2 Textures

The ground terrain uses three different texture map types loaded per planet through Three.js’s `TextureLoader`:

- a **color (diffuse) map**, applied to the material’s `map` property;
- a **normal map**, applied to `normalMap` with an amplified normal scale to emphasize surface detail;
- a **roughness map**, applied to `roughnessMap`, controlling how opaque or shiny the terrain surface appears.

All three textures are configured with repeat wrapping and a high repeat factor (150×150) so that a single texture tile can cover the large 1000×1000 unit ground plane without visible stretching, combined with 16x anisotropic filtering for sharper results at grazing viewing angles. If a texture fails to load for a given planet, the code falls back to a flat material property (e.g. a fixed roughness value) instead of leaving the material broken.

The ground geometry itself is a high-resolution plane (512×512 segments) combined with a displacement scale, allowing the flat plane to be physically deformed into three-dimensional hills rather than only appearing bumpy through normal mapping.

Beyond the terrain, the atmosphere and sky use additional procedural coloring rather than textures: a per-planet atmosphere configuration table (`atmosphereData` in `SkyboxManager.js`) defines background color, fog presence, and star color/opacity for every planet, and the starfield itself is generated procedurally as thousands of points distributed on a sphere using spherical coordinate sampling, rather than an image-based skybox.

5 User Interaction

The project implements a fairly complete first/third-person control scheme, handled jointly by `Input.js` (raw key/mouse state), `MouseControls.js` (camera look direction) and `main.js` (translating input into actual gameplay actions).

- **Look around:** moving the mouse while the pointer is locked to the canvas rotates the camera’s yaw (left/right) and pitch (up/down). Pitch is clamped to roughly $\pm 85^\circ$ to prevent the camera from flipping upside down.

- **Movement:** the W/A/S/D keys (with arrow-key fallback) move the character forward, backward, left and right relative to the current look direction.
- **Jump:** the Space bar triggers a jump, implemented with a debounce flag so that holding the key down does not repeatedly trigger new jumps before the player has landed again.
- **Throwing objects:** Throwing objects: pressing F spawns a sphere launched from the player's position. If the player is moving at the moment of the throw, the sphere is launched in the current movement direction (WASD-relative); otherwise it is launched in the direction the camera is currently facing. The throw simultaneously triggers a short arm-swing animation.
- **Camera view toggle:** pressing V switches between first-person and third-person camera modes.
- **Planet selection:** a dropdown UI element (outside the 3D canvas) lets the user choose among the nine available planets/moons. Selecting a new planet dispatches a custom `planetChanged` browser event that simultaneously updates gravity, terrain textures, sun color/intensity and atmospheric appearance, keeping every module synchronized without tight coupling between them.

All discrete actions that should only fire once per key press (jump, throw, view toggle) are protected by dedicated debounce booleans reset every frame the key is released, avoiding unwanted repeated triggering from a single continuous key-down state.

6 Animations

All animations in the project are implemented manually, inside the render loop. No animation clips or external animation data are imported.

6.1 Locomotion Animation

The character's walk cycle, implemented in `Player.animateLimbs()`, exploits the hierarchical arm/leg pivots previously described. A sinusoidal function drives the rotation of each pivot over time, with opposite phase between left and right limbs to create a natural alternating stride. Both the *cadence* (stepping speed) and the *stride amplitude* are scaled by the square root of the ratio between the current planet's gravity and Earth's gravity, so that:

- on high-gravity planets (e.g. Jupiter) steps become quicker and shorter, looking heavier and more labored;

- on low-gravity bodies (e.g. the Moon) steps become slower and wider, looking floatier.

An additional low-gravity “bounce” effect vertically displaces the torso slightly with each stride on low-gravity planets, further reinforcing the feeling of reduced weight, while this bounce disappears almost entirely under strong gravity.

Separate, non-sinusoidal poses are used when the character is airborne: a distinct arm/leg pose is applied while ascending (jumping up) versus while falling, and the torso’s vertical bounce is disabled entirely while off the ground.

6.2 Body Turning Animation

When the player moves in a direction that differs from the current camera yaw (for example strafing while looking forward), the torso is smoothly rotated (through frame-rate independent linear interpolation, with angle differences normalized to avoid a 360-degree snapping bug) to visually face the direction of movement, then eases back to a neutral orientation once the player stops.

6.3 Throwing Animation

Pressing the throw key triggers a short, timed override of the right arm’s rotation target for 0.3 seconds, producing a quick forward arm swing that plays independently of whatever locomotion animation is currently active, after which control returns automatically to the normal walk/idle pose logic.

6.4 Day/Night Cycle

On Earth, `DayNightSystem` continuously moves the sun along an orbit, and `SkyboxManager` uses the sun’s normalized height to interpolate the sky background color through three reference colors (day, dusk and night) as well as to fade the star field’s opacity in and out accordingly. This produces a fully dynamic, continuously changing lighting atmosphere purely through procedural color interpolation rather than pre-authored keyframes.

6.5 Procedural Sky Rotation

Independently of the day/night cycle, the entire procedural starfield is slowly rotated around two axes over time, giving a subtle sense of celestial motion even on planets where the day/night cycle is not active.

7 Physics and Collision System

A custom physics and collision layer was implemented to make gravity, jumping, and object throwing behave consistently and realistically across different planetary conditions, without relying on any external physics engine.

7.1 Planet’s Gravity

The **Physics** module stores the real gravitational acceleration (in m/s^2) of every supported body, ranging from the Moon’s 1.62 to Jupiter’s 24.79. The currently active gravity value is used both to integrate the player’s vertical velocity every frame and to scale gameplay parameters (movement speed, animation cadence) so that the physical differences between planets are perceivable, not just numerical.

7.2 Jumping and Ground Detection

Jumping always applies the same fixed launch velocity regardless of the current planet (representing constant muscular force from the astronaut), while gravity itself determines how high and how long the resulting jump is – naturally producing much higher, slower jumps on the Moon than on Jupiter. Ground detection is resolved against either the flat terrain or the top surface of whichever static obstacle currently sits beneath the player’s feet, allowing the player to jump on top of low obstacles and land cleanly on them instead of always falling back to ground level.

7.3 Obstacle Collision

Horizontal movement is resolved against the world’s static obstacles by treating the player as a circle on the XZ plane and testing the X and Z movement components independently, which lets the player slide smoothly along a wall’s surface when approaching it at an angle instead of being fully stopped. A secondary safety check scans for any obstacle the player’s body is still overlapping after the main resolution step, and pushes the player back to a valid position on top of it; this specifically guards against “tunneling” through obstacles at high velocities, which becomes more likely under strong gravity or after a long low-gravity jump.

7.4 Camera Collision

The **CameraManager** module prevents the camera from clipping through obstacles or terrain in both first- and third-person view. In third-person mode, the maximum upward look angle is dynamically limited so that the orbiting camera never go below the ground. Additionally, camera collision is tested not just along a single central ray from the player to the desired camera position, but along rays toward all four corners

of the camera's near clipping plane, so that an obstacle edge intersecting the periphery of the view is also correctly detected and resolved, rather than only obstacles directly in front of the camera's exact center.

7.5 Spheres Physics

Objects thrown by the player (`WeaponSystem.js`) are simulated as simple spheres subject to the current planet's gravity, bouncing off the ground with energy loss and friction, and additionally colliding elastically with the same static obstacles used for player collision: for each obstacle overlap, the axis of least penetration is computed and the projectile's velocity is reflected along that axis with partial energy loss, producing a believable bounce off cubes rather than passing through them.